← Portfolio

PROJECT 2: FUN WITH FILTERS AND FREQUENCIES

1.1

Here's the convolution implementation with 4 loops. We first ensure that our kernel is 2-dimensional, and it has odd heights and widths to ensure the convolution is symmetric. We implement same padding; this requires padding the image before convolution with the kernel height or width, subtracted by 1, then integer-divided by 2. This makes sense; with a 3x3 kernel, we want to add 1 extra space on each side of the image so that our image ends up being the same side once the convolution is done. That way, the first convolution will be on the square with diagonal (0, 0) to (2, 2), centered at (1, 1) on the padded image → which is (0, 0) on the initial image.

For convolution, we must flip the kernel. Then, we essentially iterate through each image pixel (with i for height and j for width). At each image pixel, we apply the kernel by iterating through the kernel height and kernel width, and adding the pixel value there scaled by the corresponding flipped kernel value to a running total. This running total, after the full dot product operation, gets placed on our final_image, which starts out as a 2D array of 0s but is filled after the operation.

This implementation on the image of me was relatively slow, at 5.8513s.

```python
def convolve4loops(img, kernel):
    assert kernel.ndim == 2 and kernel.shape[0] % 2 == 1 and kernel.shape[1] % 2 == 1
    height_padding = (kernel.shape[0] - 1) // 2
    width_padding = (kernel.shape[1] - 1) // 2
    height = img.shape[0]
    width = img.shape[1]
    padded_img = np.zeros((height + 2*height_padding, width + 2*width_padding))
    padded_img[height_padding:height_padding+height, width_padding:width_padding+width] = img
    flipped_kernel = kernel[::-1, ::-1]
    final_img = np.zeros((height, width))
    for i in range(height):
        for j in range(width):
            running_total = 0
            for k in range(0, kernel.shape[0]):
                for l in range(0, kernel.shape[1]):
                    running_total += padded_img[i+k, j+l] * flipped_kernel[k, l]
            final_img[i, j] = running_total
    return final_img
```

CONVOLVE4LOOPS

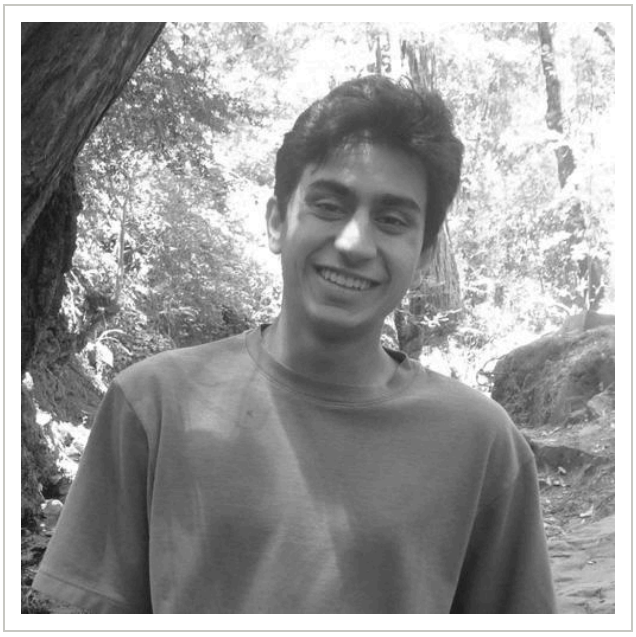
The convolve function with 2 loops is largely the same, except we use vectorization for our dot product. Once we have the i, j corresponding to the pixel we'd like to apply the kernel to, we directly do the dot product on the corresponding range of the padded image with our kernel. This saves a substantial amount of time, at 0.5747s.

```python
def convolve2loops(img, kernel):
    assert kernel.ndim == 2 and kernel.shape[0] % 2 == 1 and kernel.shape[1] % 2 == 1
    height_padding = (kernel.shape[0] - 1) // 2
    width_padding = (kernel.shape[1] - 1) // 2
    height = img.shape[0]
    width = img.shape[1]
    padded_img = np.zeros((height + 2*height_padding, width + 2*width_padding))
    padded_img[height_padding:height_padding+height, width_padding:width_padding+width] = img
    flipped_kernel = kernel[::-1, ::-1] # for convolution
    final_img = np.zeros((height, width))
    for i in range(height):
        for j in range(width):
            final_img[i, j] = (padded_img[i:i+kernel.shape[0], j:j+kernel.shape[1]] * flipped_kernel).sum()
    return final_img
```

CONVOLVE2LOOPS

The scipy implementation is the quickest, at only 0.0248s. Lots of optimization! I use same padding, and I fill the boundary with 0s to match our implementation. I also verify the correctness of my implementation by convolving an image of myself with a 9x9 box kernel and verifying that all three implementations produce the same output image (with np.allclose).

Here are the three output images.



ORIGINAL (GRAYSCALE)



9X9 BOX · 4 LOOPS · 5.8513S



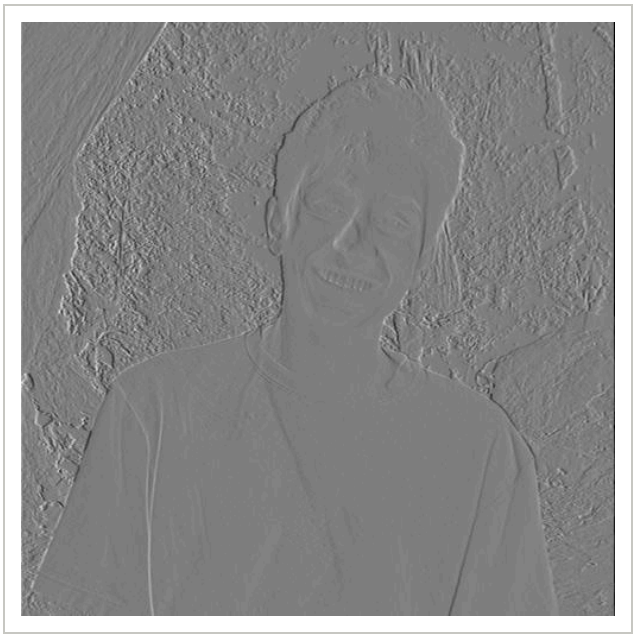
9X9 BOX · 2 LOOPS · 0.5747S



9X9 BOX · SCIPY · 0.0248S

We see that each of them have a slight black blur around the edges/border. This is due to the fact that we pad with 0s, so the convolution around the edges with the box kernel results in lower-magnitude pixel values.

We also convolve with the derivative filters. We can see D_x recovers vertical edges, and D_y recovers horizontal edges!



D_X



D_Y

1.2

Here's the cameraman image.

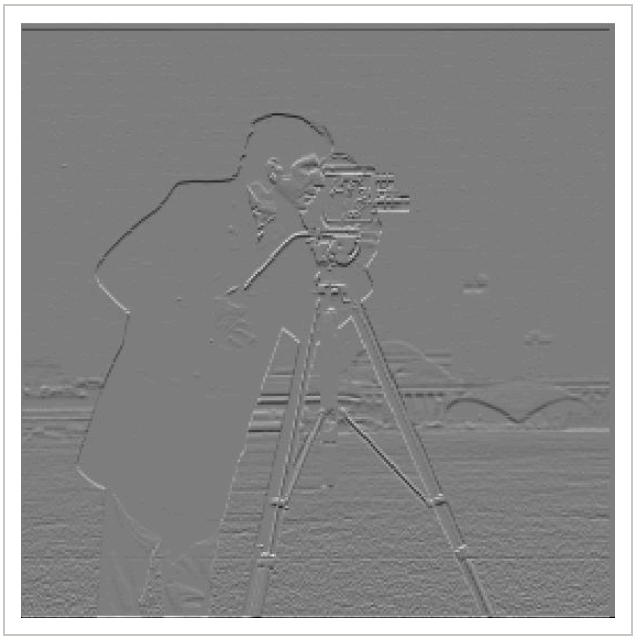


CAMERAMAN

First, we find the partial derivatives, noticing a similar pattern to above.



CAMERAMAN · D_X



CAMERAMAN · D_Y

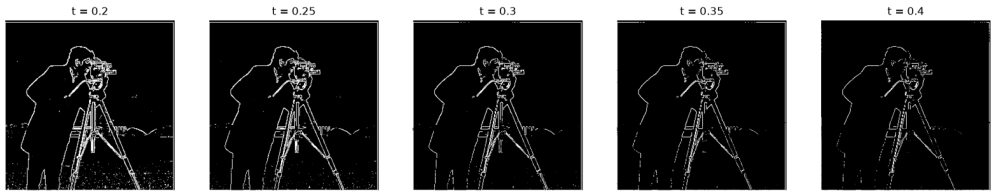
To find the gradient magnitude image, we take the pixel values from cameraman_D_x and cameraman_D_y (the two partials), square them, add elementwise, and take the square root. We see it identifying rough edges!



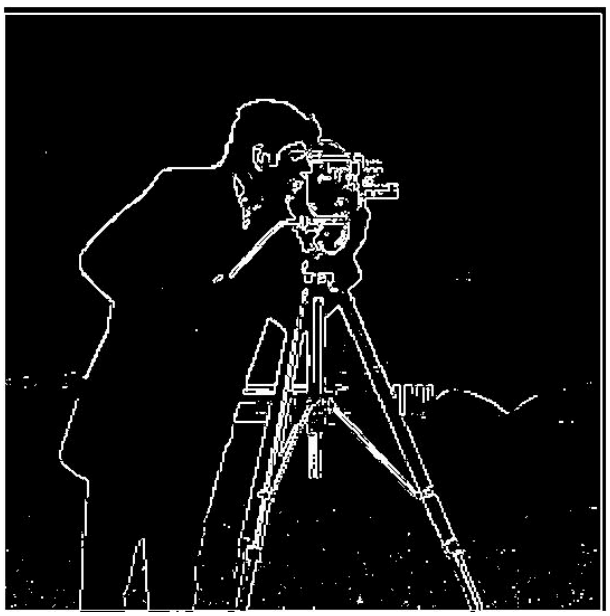
GRADIENT MAGNITUDE

By binarizing the image by setting pixels meeting a certain threshold to 1 (and the others to 0), we can get sharper, more defined edges. However, identifying this threshold requires testing candidates; too small, and we'll capture noise. Too large, and we'll lose meaningful edge

signal. I sweep over an initially small range before narrowing it down and choosing my final threshold of 0.25, which I found struck the best balance, tolerating some noise in the background to minimize the loss of the cameraman's outline.



THRESHOLD SWEEP



EDGES · THRESHOLD 0.25

1.3

The results in 1.2 were still noisy. To correct this, I use a Gaussian kernel. For all Gaussian kernels in this project, I use a fixed sigma corresponding to the kernel size by the formula (s - 1) / 6, where s is the kernel height/width (kernel is square). Gaussian filters create blurs, as we see below.

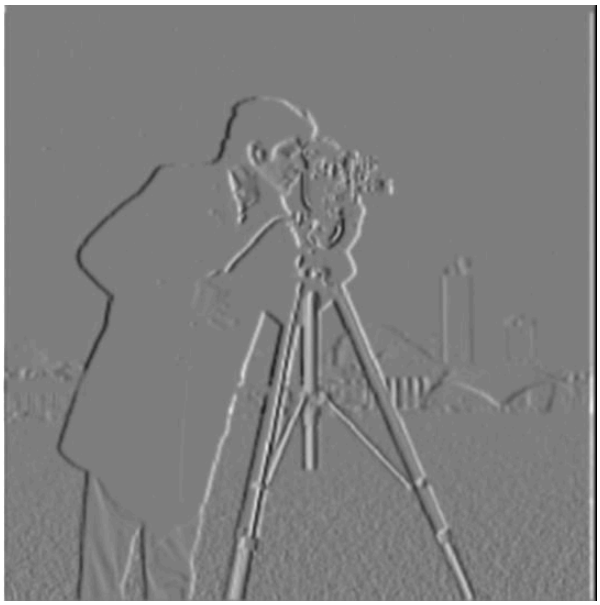


ORIGINAL

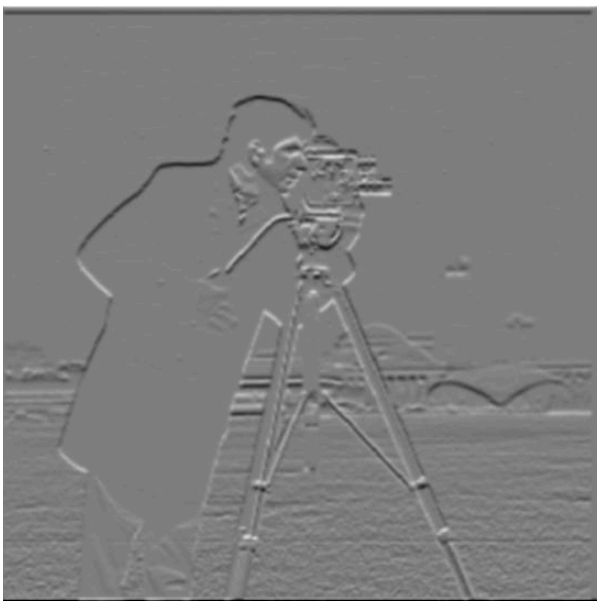


BLURRED

I can then convolve the image with our derivative filers, compute the gradient magnitude image, and finally find a threshold for the best image.



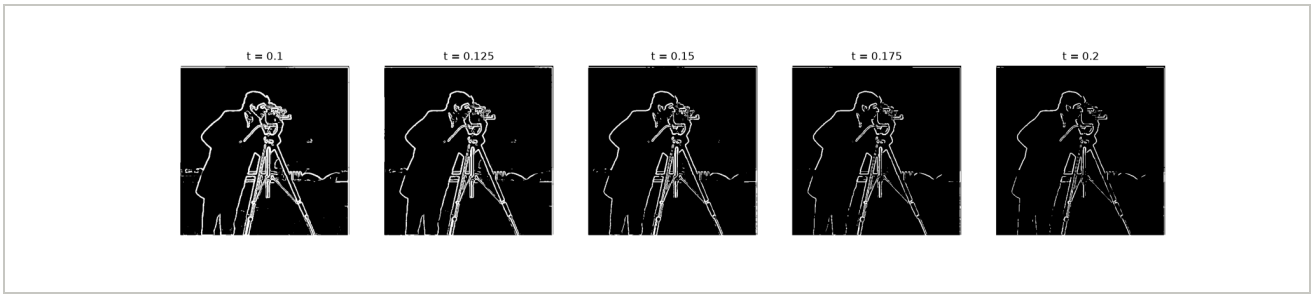
BLURRED · D_X



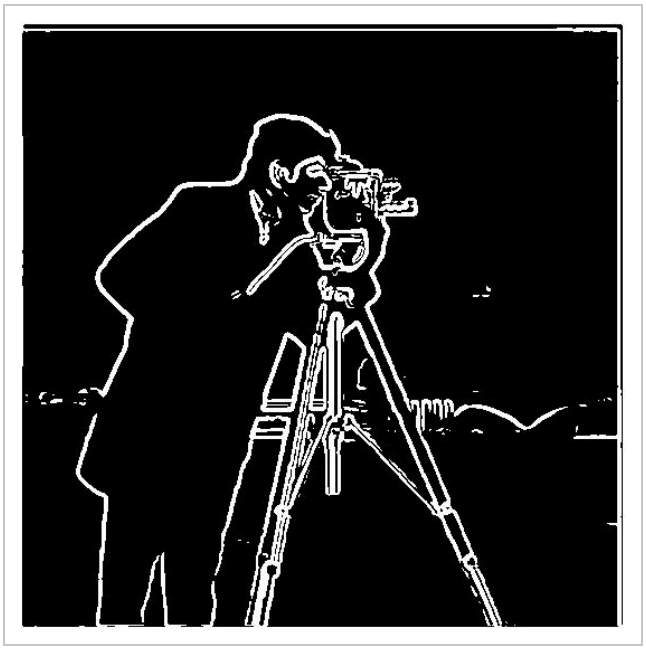
BLURRED · D_Y



GRADIENT MAGNITUDE

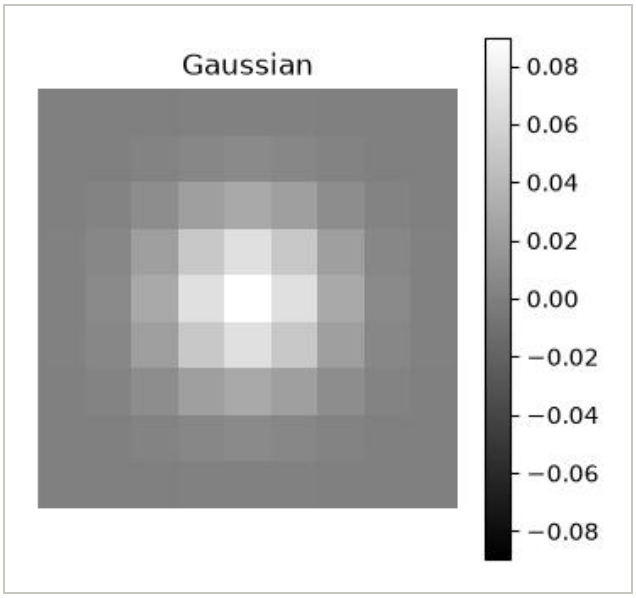


THRESHOLD SWEEP

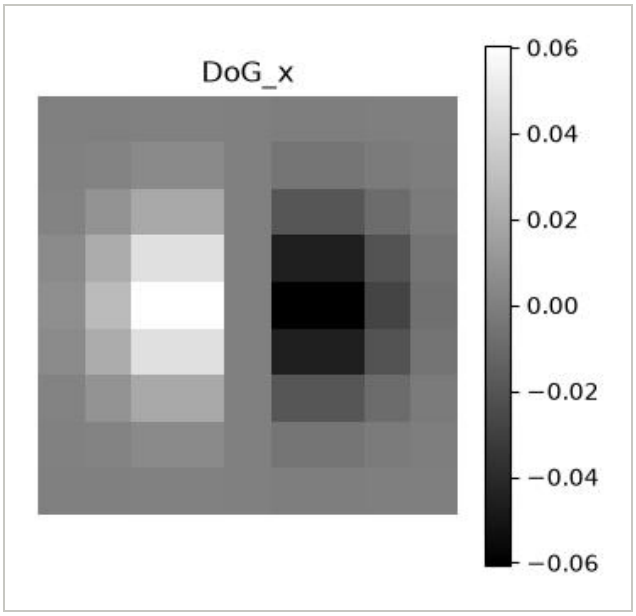


EDGES · THRESHOLD 0.125

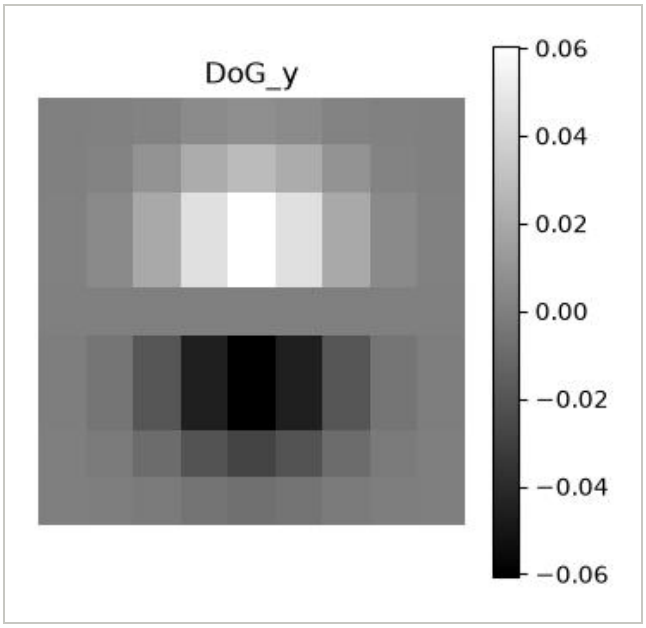
Alternatively,weI can do the same thing with a single convolution by convolving the Gaussian with out derivative filters. Here are the resulting filters visualized after the convolution (I use same padding as well for consistency).



GAUSSIAN

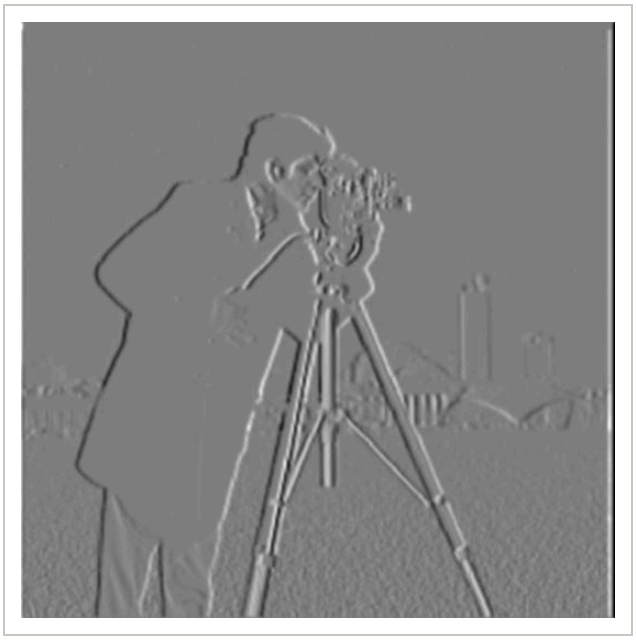


DOG_X

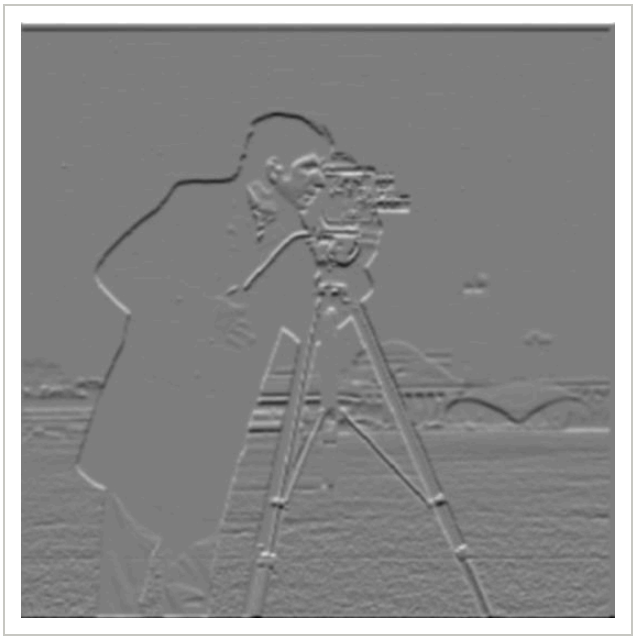


DOG_Y

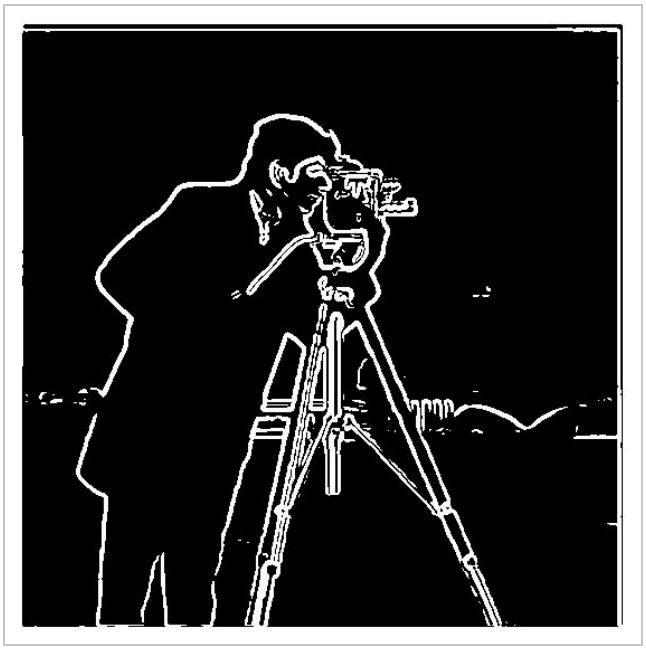
As we can see, DoG_x recovers vertical edges, and DoG_y recovers horizontal edges. We can then convolve the original cameraman image with both these filters. We get the same results as above.



DOG_X



DOG_Y



EDGES · THRESHOLD 0.125

2.1

A low-pass, or blur, filter applies a Gaussian kernel to the image. Then, if we subtract this low-frequency, blurred image from the original image, we recover the sharp details that were initially filtered out. This is our high-frequency image; it keeps only those details that were smoothed out by the low-pass filter. We can apply this to the Taj Mahal.



ORIGINAL



LOW-PASS



HIGH-PASS

We can then take this difference and add it back to our original image with some weight alpha. This gets us the unsharp mask filter. Nice properties of convolutions (namely, that when you convolve an image with the impulse, you get the same image, along with commutativity, associativity, and distributivity) let us condense the unsharp mask filter into one kernel as below, where f is the original image, g is the gaussian kernel, e is the impulse, and alpha is the weight we're giving to our high-frequency image. (This equation is taken from the class slides)

$$f + \alpha(f - f * g) = (1+\alpha)f - \alpha f * g = f * ((1+\alpha)e - \alpha g)$$

As alpha increases, i.e. as we increase the amount of the high-pass filtered image we add, the image becomes a lot sharper and grainier. We can see this very clearly with the Taj Mahal image.



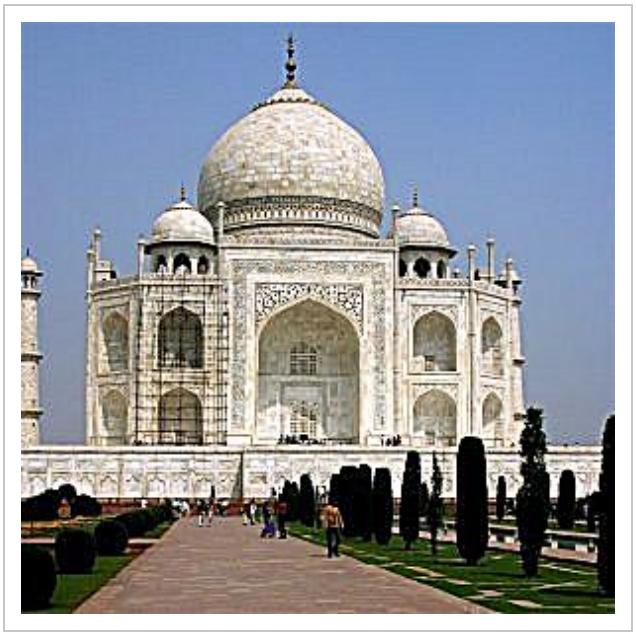
ALPHA = 0



ALPHA = 0.5



ALPHA = 1



ALPHA = 2



ALPHA = 5



ALPHA = 10

I also did this on a little festive image of myself. I had to go up to alpha = 10, but you can very clearly see the sharp detail, particularly on my messy hair.



ALPHA = 0



ALPHA = 0.5



ALPHA = 1



ALPHA = 2



ALPHA = 5



ALPHA = 10

2.2

For the hybrid images, one image receives the low-pass filter, and the other receives the high-pass filter, following the practice in Oliva et al. The cutoff frequency of each filter had to be

calibrated to for each pair; I calibrated this frequency by adjusting the sigma value of the Gaussian kernel. To compensate, this meant that we had different Gaussian kernel sizes so I could keep the (s-1)/6 ratio. In this case, we compute the kernel size from the sigma, 2 * (3 * sigma) + 1 so the height/width is still odd. I record sigma values with the images; generally, the low-frequency image needed a sigma that was 3-8 units higher than the high-frequency image.

We produce three sets of hybrid images. Here are the first two.



PUFFIN · LOW-PASS · SIGMA 15



BASEBALL · HIGH-PASS · SIGMA 7



HYBRID



DEREK · LOW-PASS · SIGMA 15



NUTMEG · HIGH-PASS · SIGMA 7



HYBRID

For the third, I'll show the entire process. We first align the ends of the banana and telephone.





TELEPHONE · ORIGINAL



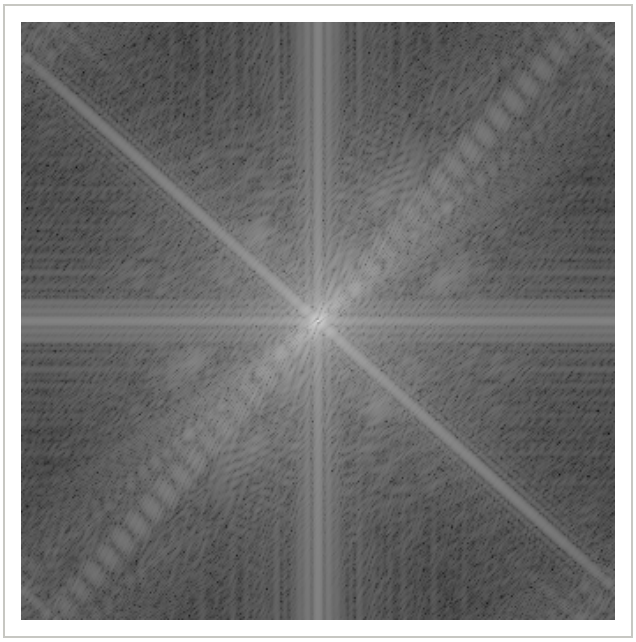
BANANA · ORIGINAL



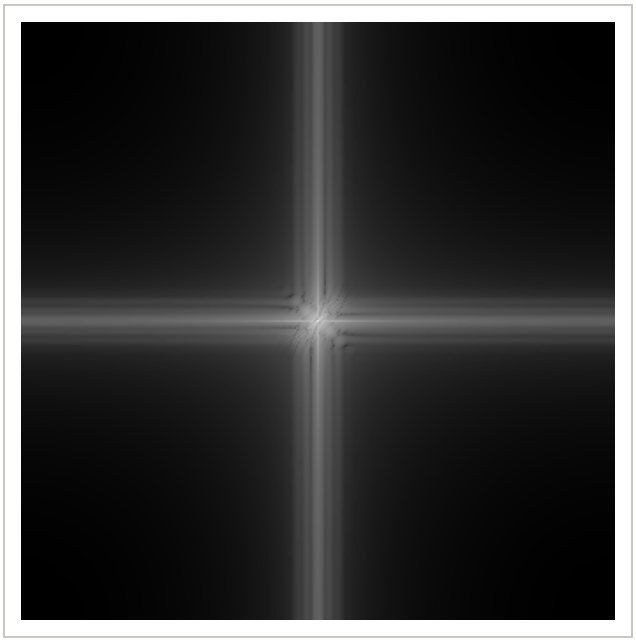
TELEPHONE · ALIGNED

BANANA · ALIGNED

When we apply the low-pass filter to the telephone, we see that circular mask that keeps the central bright region and darkens out the extremities of the image. I chose a sigma of 7 here; I initially had a higher sigma, but the telephone was not recognizable even from a far distance, so I had to reduce it.

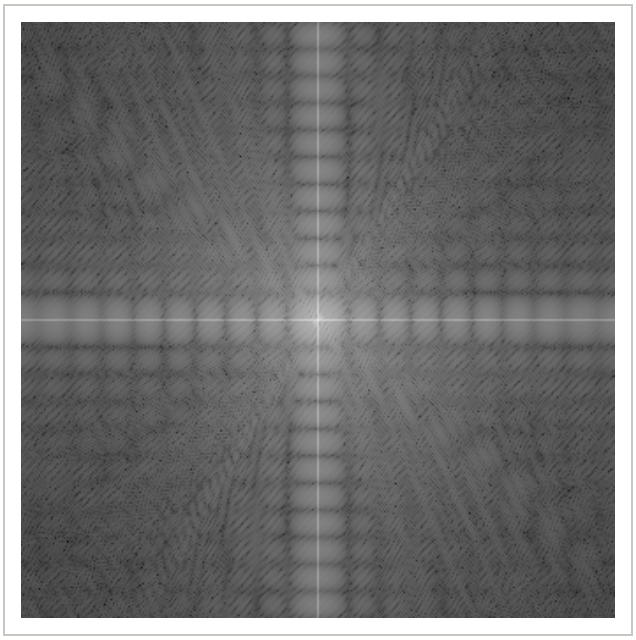


TELEPHONE · FFT

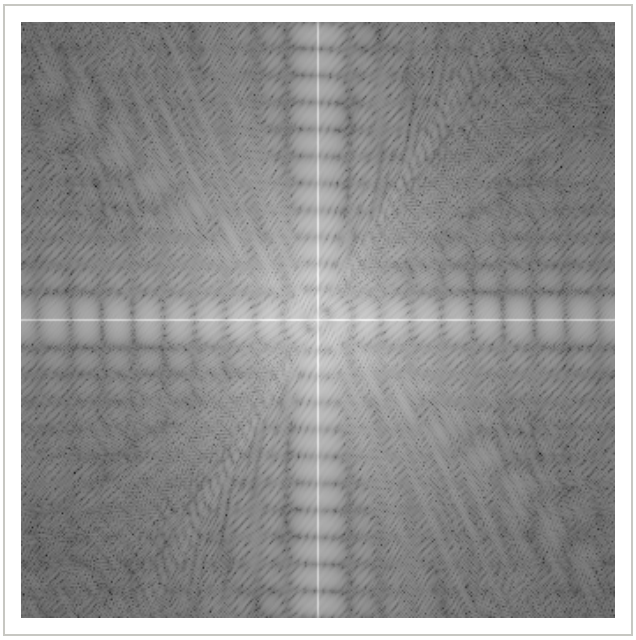


TELEPHONE · LOW-PASS FFT · SIGMA 7

In contrast, when we apply the high-pass filter to the banana, we see the reverse effects; the extremities of the image brighten, and the center gets slightly darker. But we can still clearly see it's the same image with the distinctive pattern on the vertical and horizontal. I chose a sigma of 4 here; this was balanced relative to the choice of the telephone to ensure the banana faded away at a distance but was still clear up close.

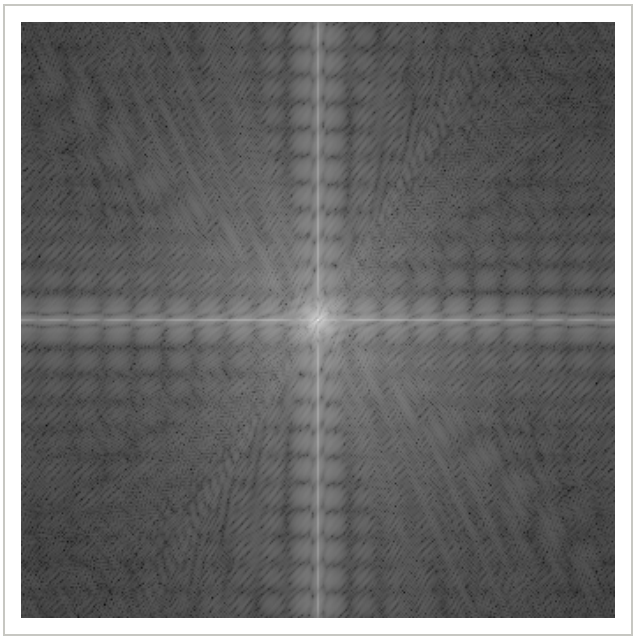


BANANA · FFT



BANANA · HIGH-PASS FFT · SIGMA 4

The Fourier transformation of the image sum reflects the low-frequency patterns of the telephone (namely, the bright center and verticals/horizontals) and the high-frequency patterns of the banana (the strobe pattern).



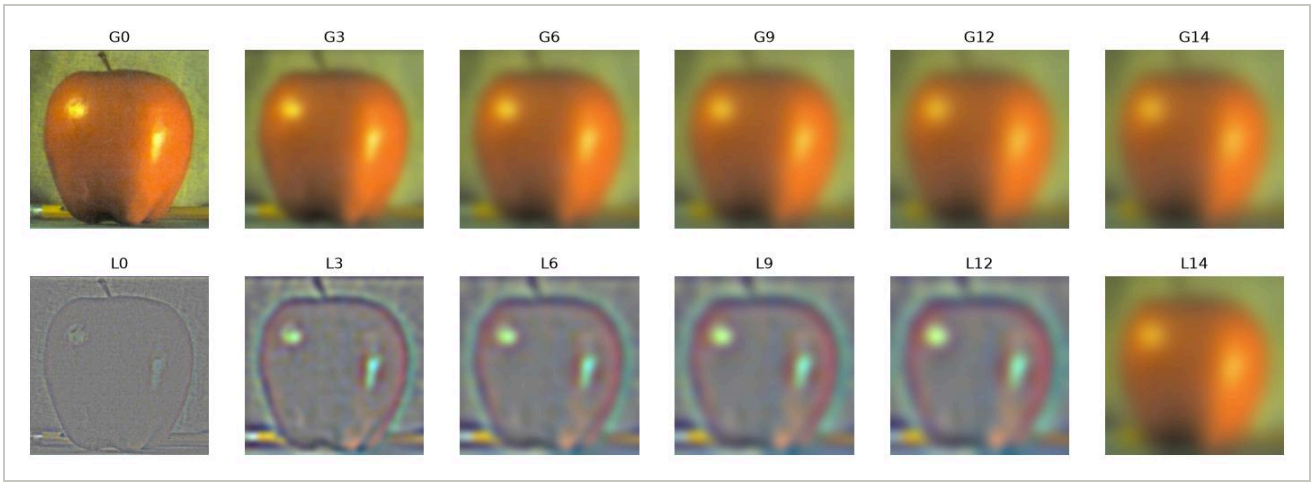


HYBRID · FFT HYBRID

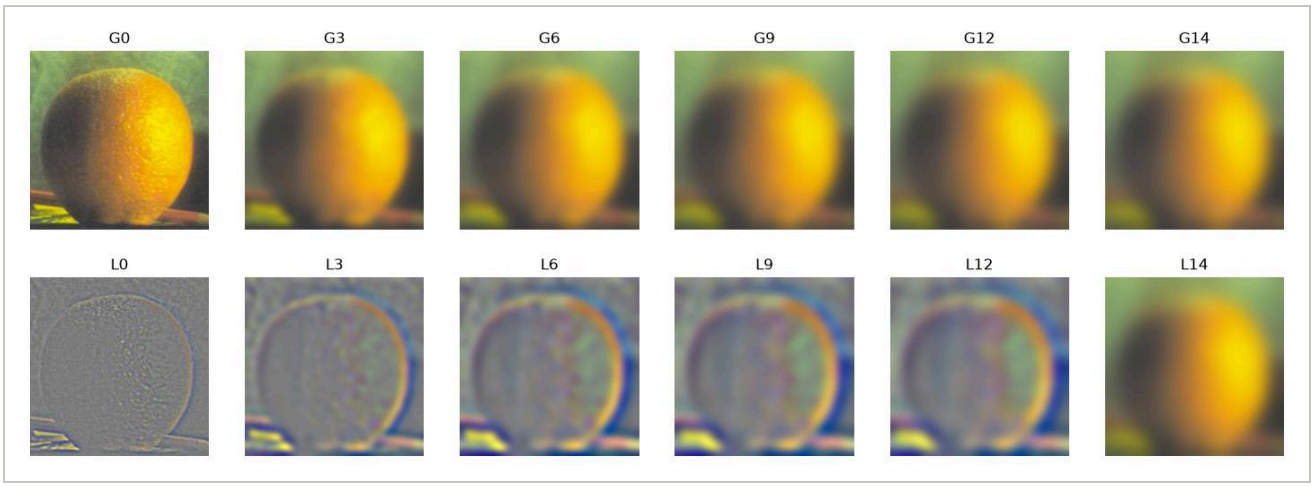
2.3/2.4

First, I implement a Gaussian and Laplacian stack. For the Gaussian stack, I choose a constant kernel size and sigma, and continually apply the filter. I.e. G[i] = G[i-1] * (gaussian filter). For the Laplacian stack, I iterate through the Gaussian stack, such that L[i] = G[i] - G[i+1]. For L[0], this is the original image minus the blurred image, to get the resulting details. The last term of the Laplacian stack is just the last term of the Gaussian stack, i.e. L[i] = G[i].

We can plot out the Gaussian and Laplacian stacks for the orange and apple here. I use a kernel size of 23 and 15 levels of the stack, chosen through just experimentation for good blending. For the sake of space, I only plot increments of three, along with the last element of the stack.



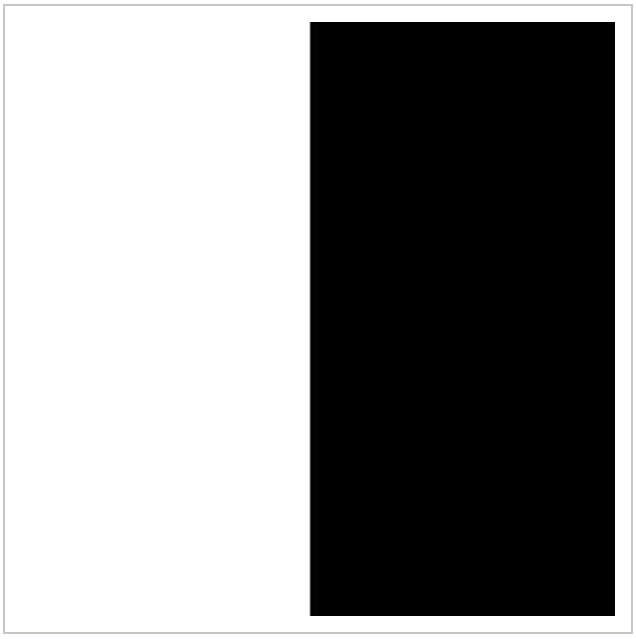
APPLE · GAUSSIAN (TOP) AND LAPLACIAN (BOTTOM) STACKS



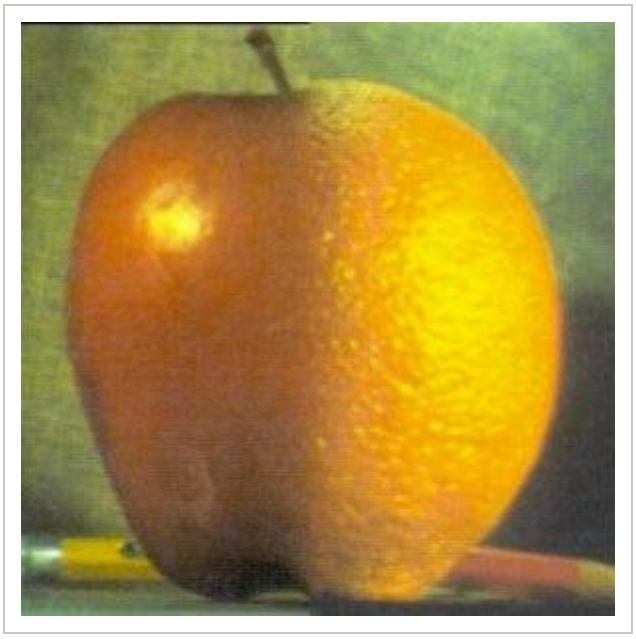
ORANGE · GAUSSIAN (TOP) AND LAPLACIAN (BOTTOM) STACKS

Then to blend, we first create a mask. We can construct a variety of custom masks based on an image, or just a straightforward vertical or horizontal mask by simply initializing an array of size image_height x image_width to 0, and setting the left/right half or top/bottom half to 1. I use an implementation similar to align_images to choose a polygonal mask for each image, so we can construct both these vertical and horizontal masks but also any custom nonlinear (but straight-edged) one.

Once we make a mask, we must build a Gaussian stack. Then, to reconstruct, we multiply the Gaussian-transformed mask at index/level i with the Laplacian at the corresponding index for the first image (the one we want to be nonzero for the map). This gives us, intuitively, the part of the aspects of the first image we want to keep, at the frequency corresponding to index i. Then similarly, we multiply 1 - the Gaussian-transformed mask at the same level with the Laplacian for the second image, to extract the aspects of the second image we want to keep. These are added together to form the blended image for that level. If we sum the blended image at all levels, we get the combined image! This makes sense, as at the lowest level, the element in the Laplacian stack reduces to the element in the Gaussian stack, so we're combining two low-frequency images. Then we add the details captured by the Laplacian at each corresponding level.



MASK



ORAPLE

We can visualize the full process with the figure below, recreated to match the results in Figure 3.42.

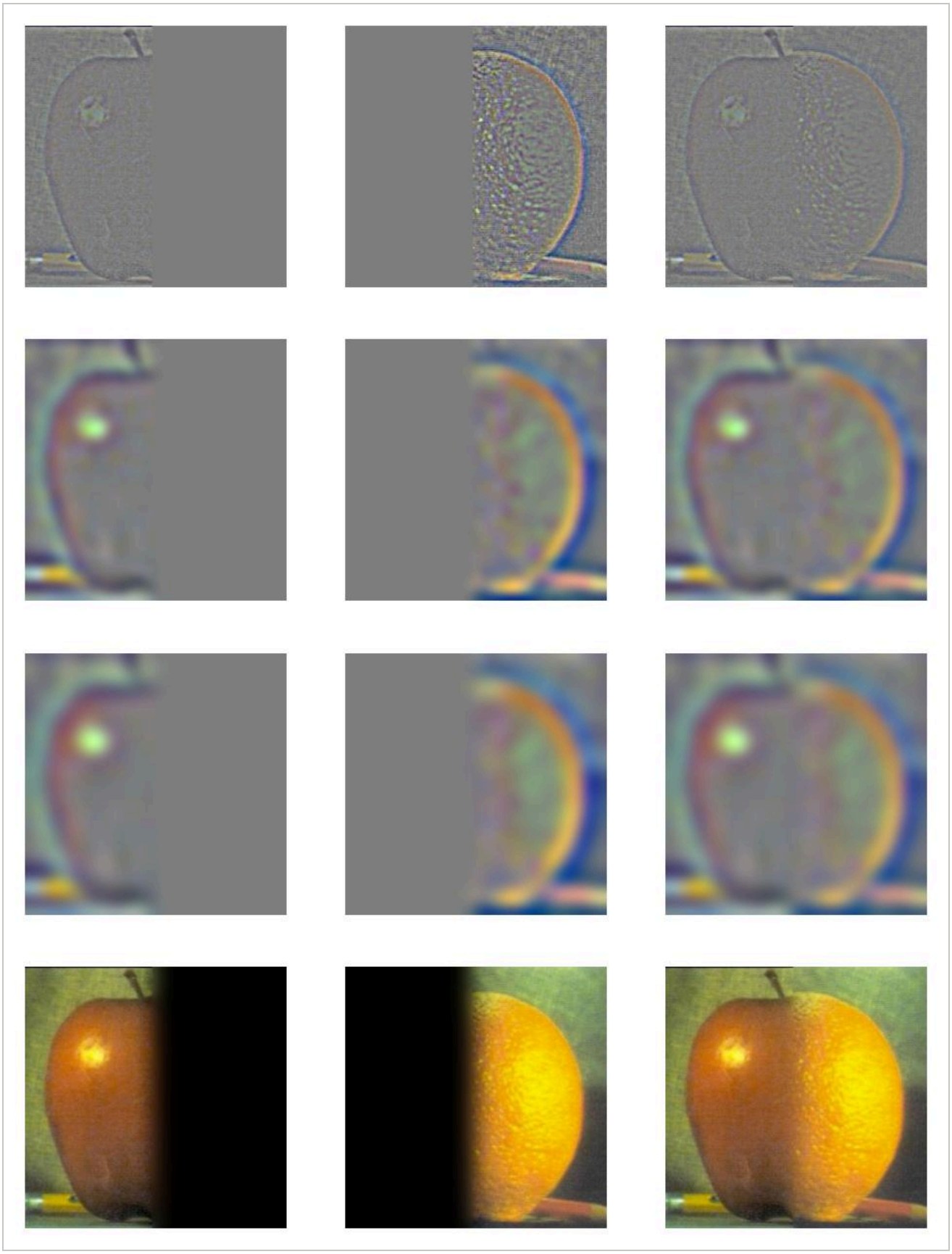
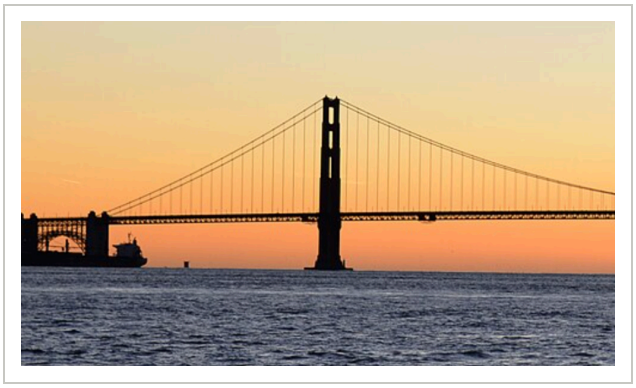


FIGURE 3.42 RECREATION · LEVELS 0, 5, 10, AND THE FULL SUM

We can also do the same process with two other sets of images.

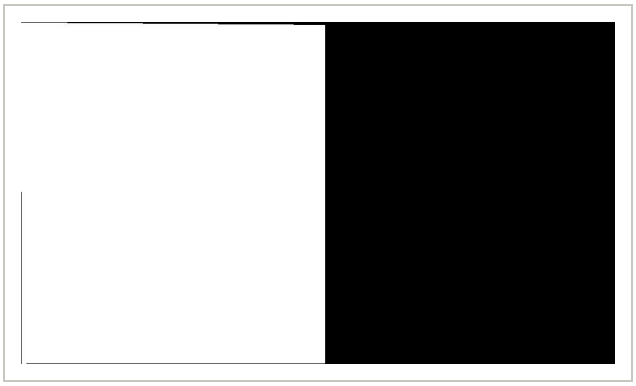
First, I tried connecting the Golden Gate and the Bay Bridge. This was difficult, as there were many areas where I wanted these bridges to connect, but each has a different aspect ratio. I ultimately chose to ensure they connected at the top and the middle road, sacrificing the bottom pillar and ocean connection. But it's still pretty cool! I just used a simple vertical mask. All figures are below.



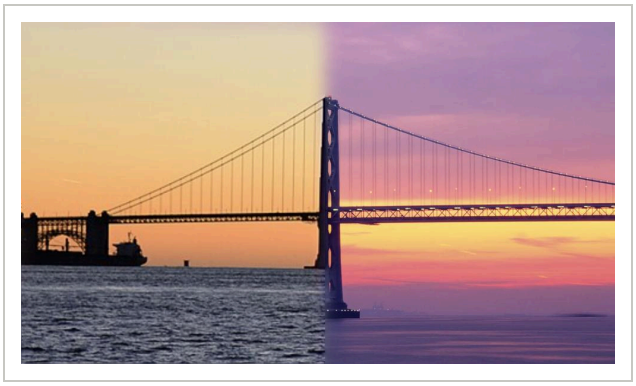
GOLDEN GATE



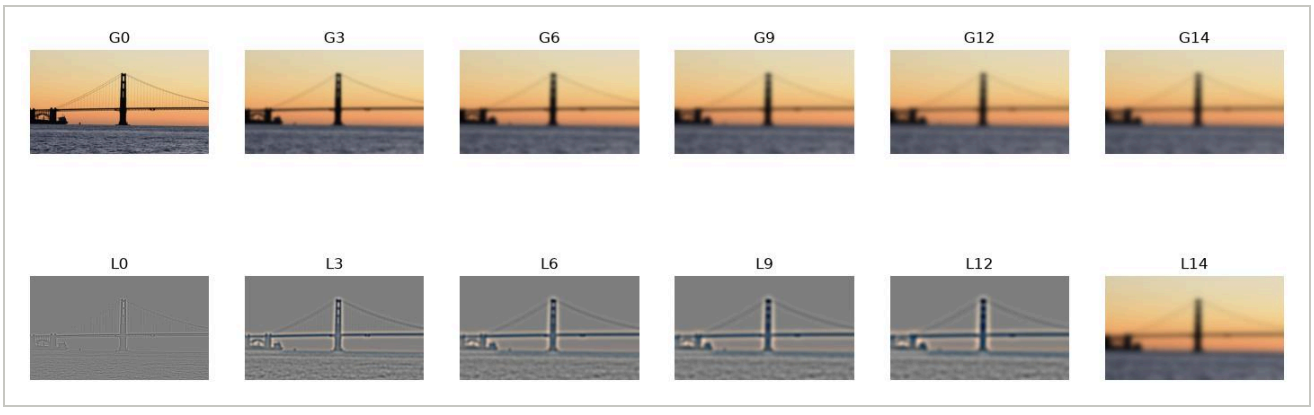
BAY BRIDGE



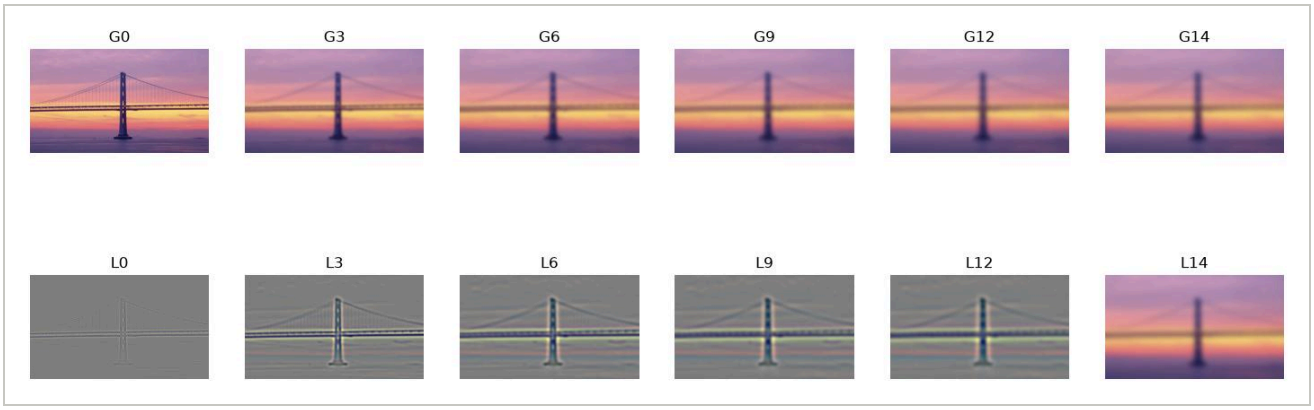
MASK



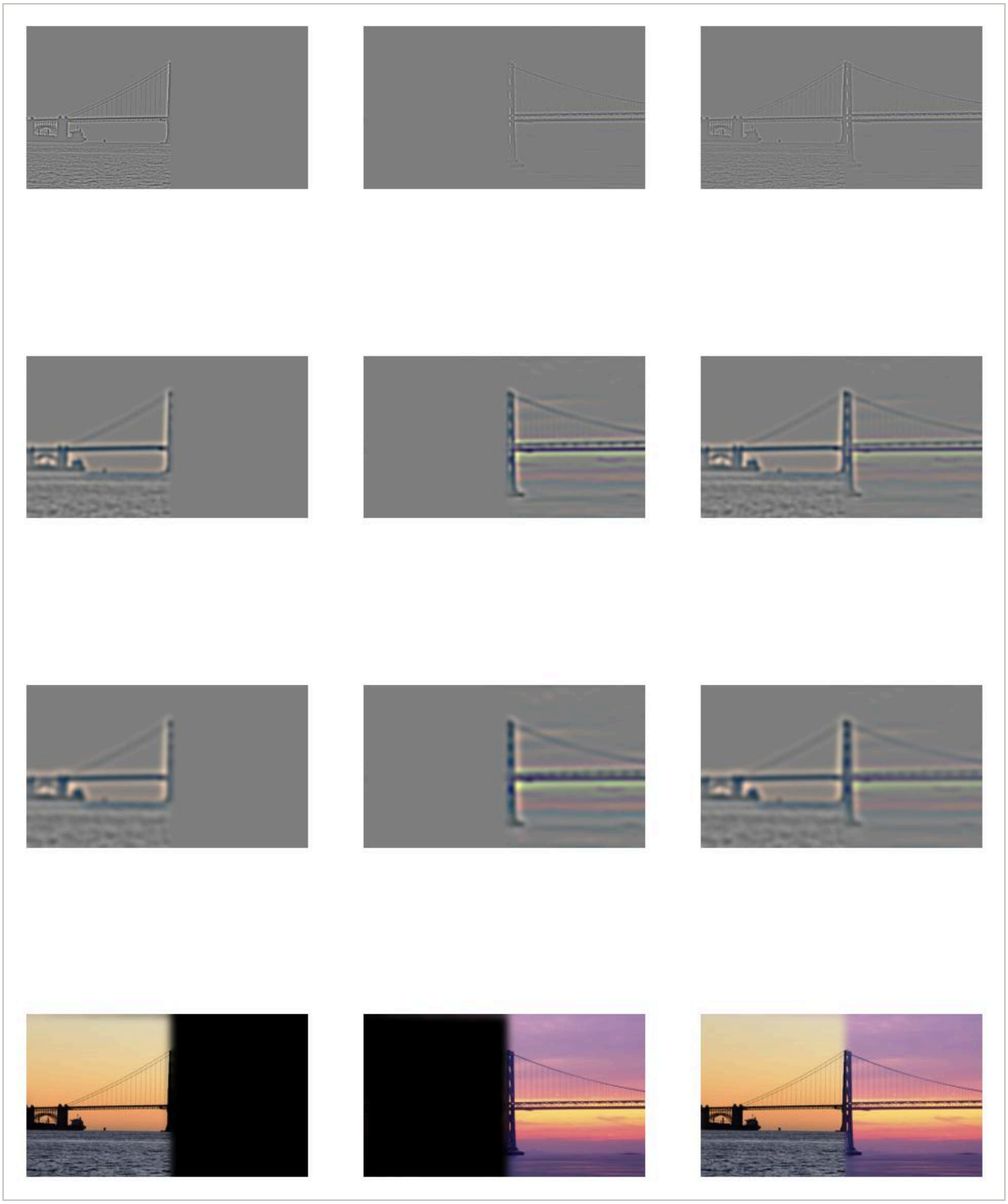
BLEND



GOLDEN GATE · GAUSSIAN AND LAPLACIAN STACKS



BAY BRIDGE · GAUSSIAN AND LAPLACIAN STACKS



BLENDING BY LEVEL

Then second, for the nonlinear mask, I had an image of a traffic cone on the street and an ice cream cone. I replaced the traffic cone with the ice cream cone, and used a triangular mask to do so. This was difficult; because this method is built for blending—and not necessarily sharp

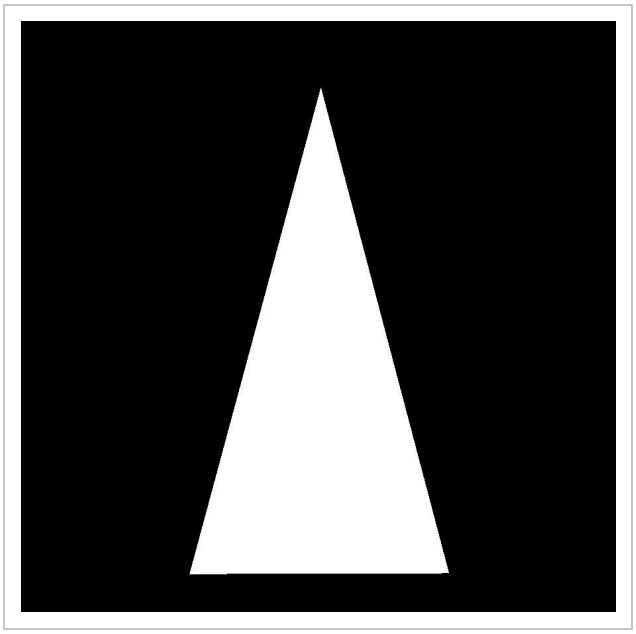
edges—the pink background of the ice cream cone image bled a little bit into the street of the traffic cone.



ICE CREAM CONE



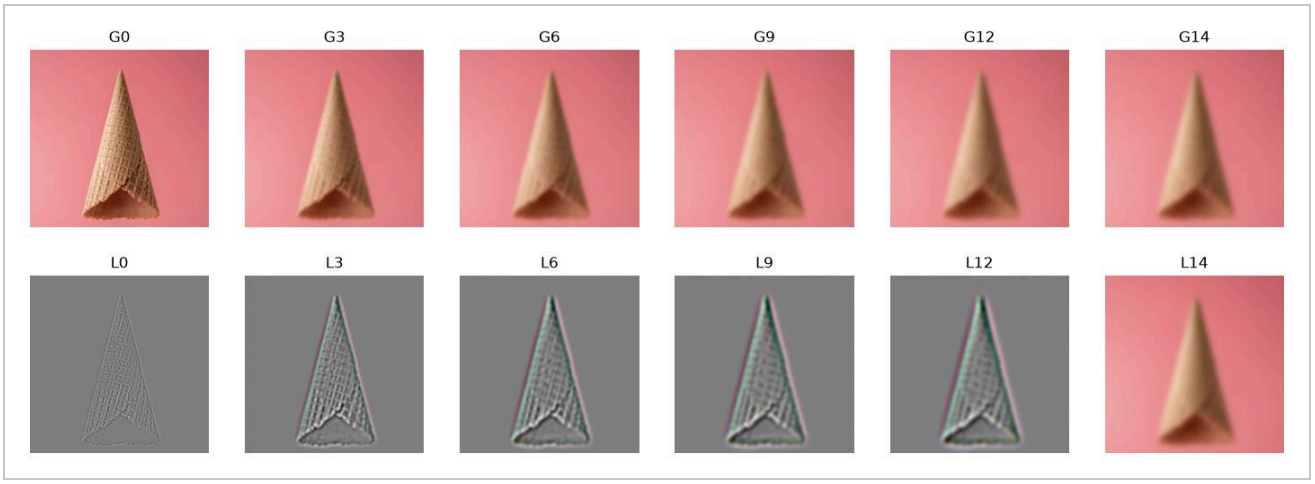
TRAFFIC CONE



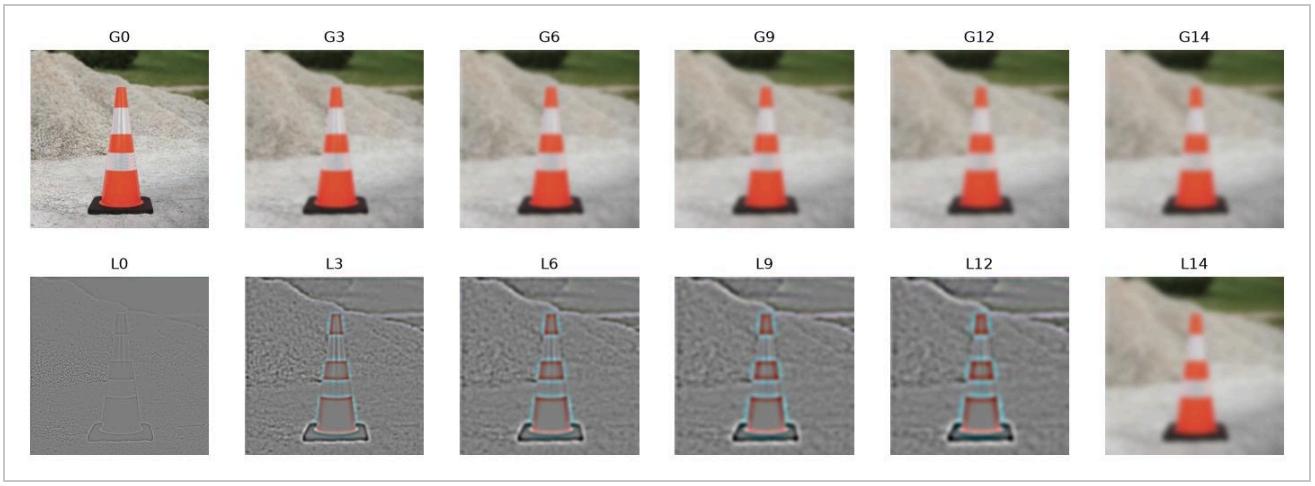
MASK



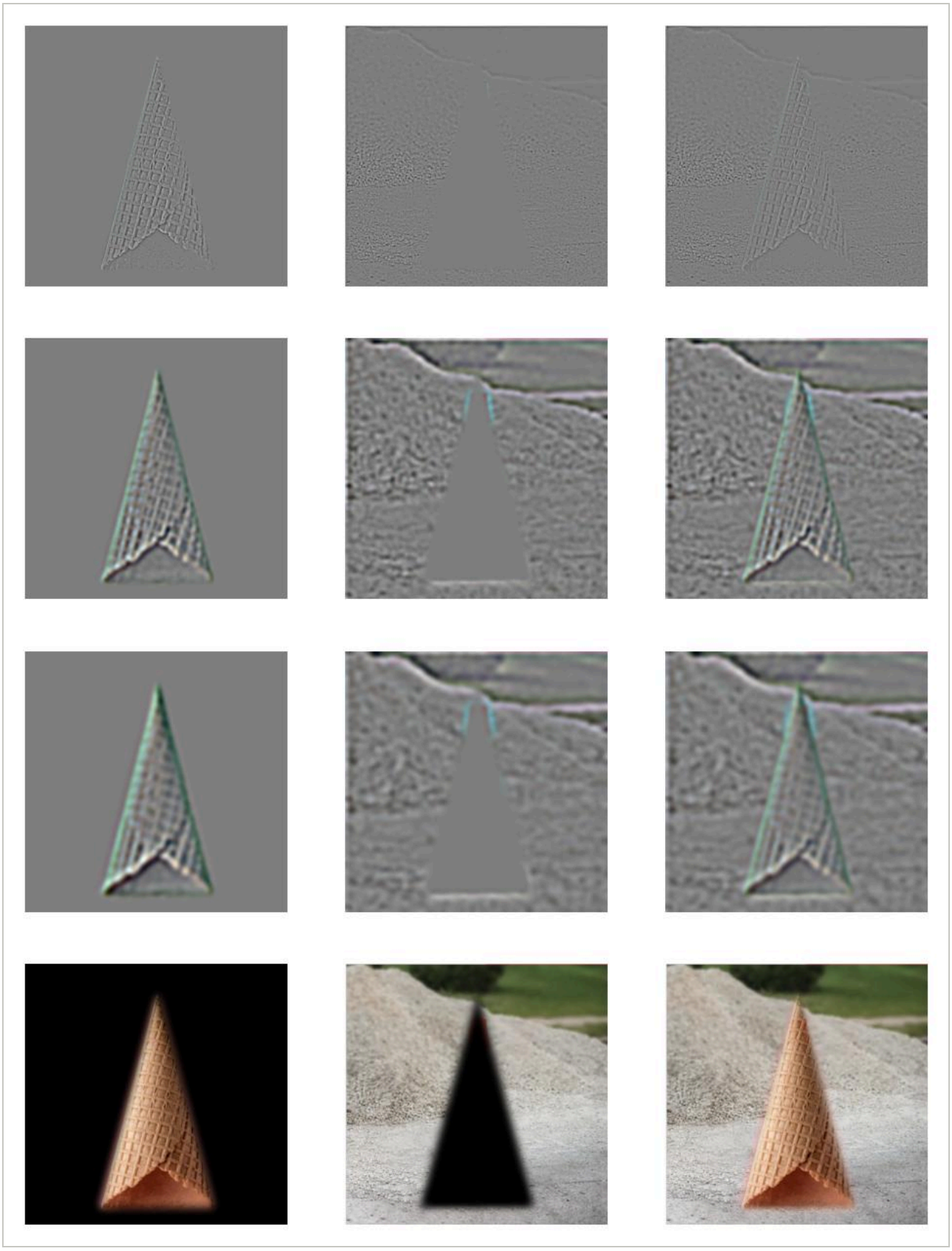
BLEND



ICE CREAM CONE · GAUSSIAN AND LAPLACIAN STACKS



TRAFFIC CONE · GAUSSIAN AND LAPLACIAN STACKS

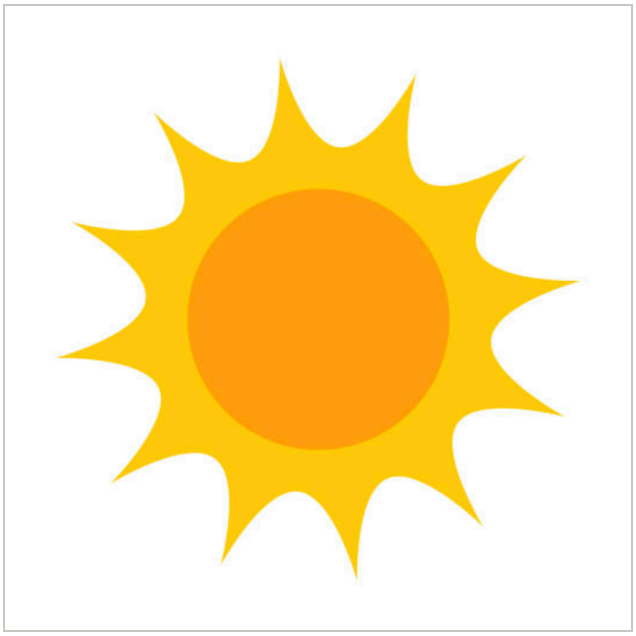


BLENDING BY LEVEL

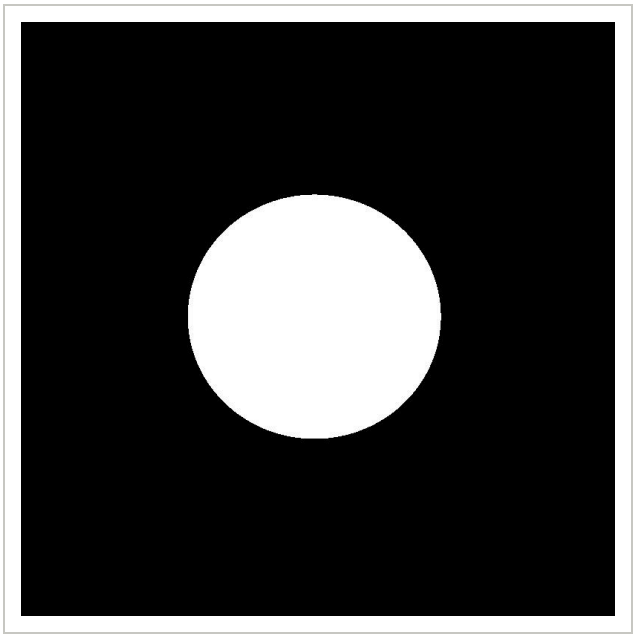
Finally, I realized that the nonlinear mask couldn't just include straight lines, so I did a third hybrid! I created a custom elliptical mask. I modified my make_mask function to take a parameter to create an ellipse, defined by the centers and major/minor axes (essentially radii), and applied it to create a dazed sun. You essentially plot the two corners that the ellipse gets bounded by, and the mask is created. Results below!



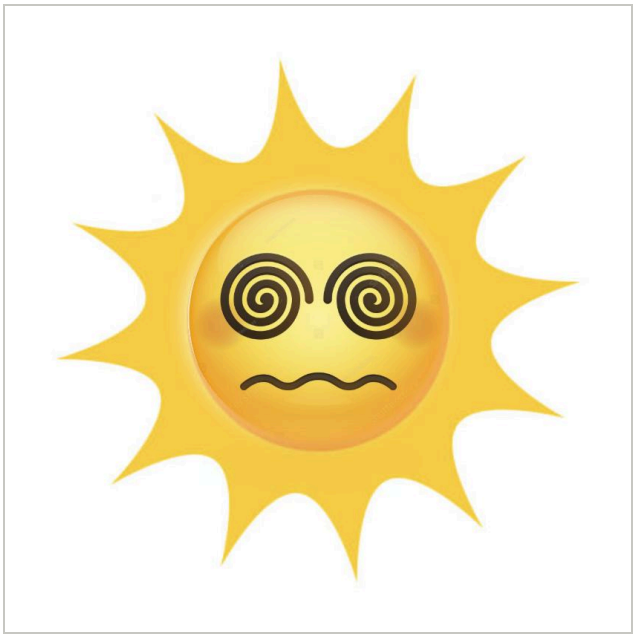
DAZED



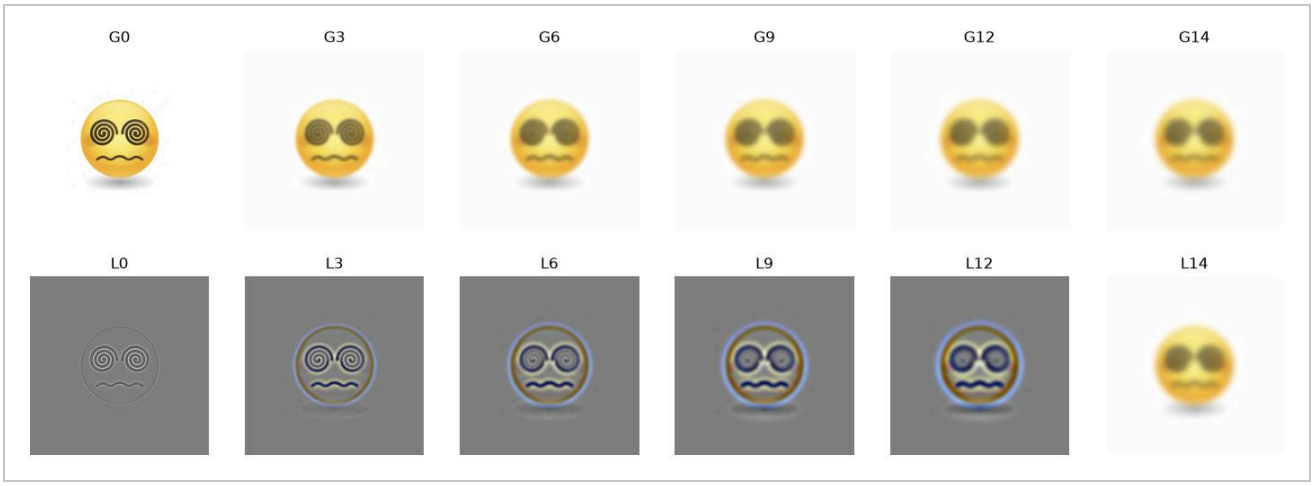
SUN



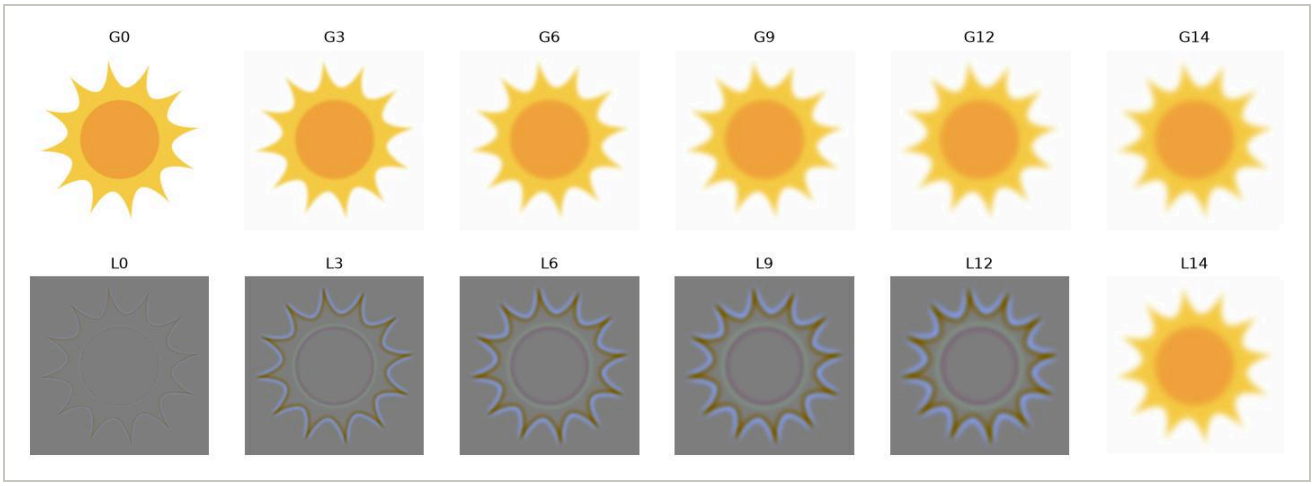
MASK



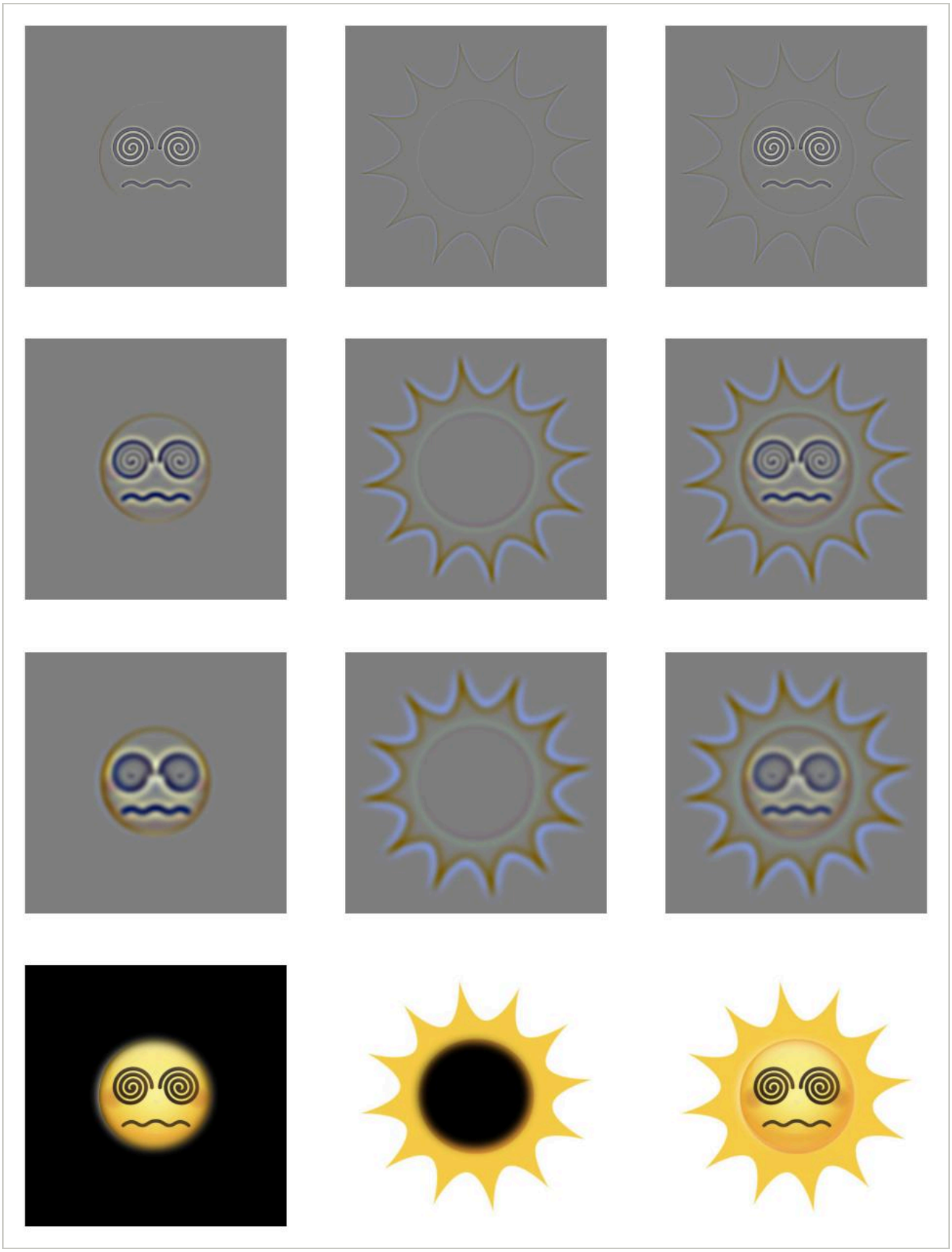
BLEND



DAZED · GAUSSIAN AND LAPLACIAN STACKS



SUN · GAUSSIAN AND LAPLACIAN STACKS



BLENDING BY LEVEL

As a final reflection: the most important thing I gained from this project was intuition with the low-pass and high-pass filters in relation to hybrid images. Calibrating the sigmas and choosing what image to be low versus high frequency helped me understand how decreasing and increasing the frequency directly influences how you see and perceive the image up close and far away. And with the high-pass filter, I was able to get a better understanding of why this difference operation actually works. Before, it seemed to just come out of left-field, particularly because it's a difference operation that seems not to retain the most important parts of the image! But it's interesting to see how much detail (particularly in the baseball case!) is actually extracted and amplified in the high-frequency case.